LogDx-CI: Benchmarking Log Reduction Tools for LLM Root-Cause Diagnosis

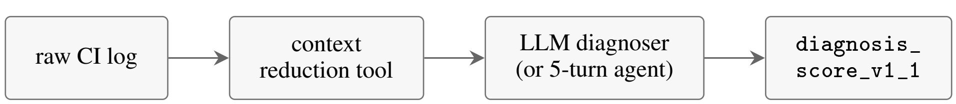
Bowen Qin
National University of Singapore
github.com/eyuansu62/LogDx

Abstract

CI failure logs are large (median 5k lines, max 200k in this corpus) and noisy. Coding agents that try to debug them depend on an upstream tool to reduce the log to a manageable context, but the field has had no public empirical comparison of which reductions preserve enough evidence for downstream LLM diagnosis. We introduce **LogDx-CI**, a benchmark that compares **11 context-reduction tools** (raw, tail, grep, three RTK modes, two real LLM map-reduce summarizers, three hybrid routers) on **35 real GitHub Actions failure cases**, scored by **3 LLM debugger families** (Claude Haiku 4.5, Claude Sonnet 4.6, OpenAI gpt-5-mini) plus a Sonnet 4.6 tool-using agent. We report three load-bearing findings. (1) Hybrid grep+tail routers dominate the cost-quality Pareto frontier; the top two methods score 0.670 / 0.666 at ~$0.03 per case, same-ballpark quality as standalone grep at $4.5\times$ fewer tokens. (2) In the agent-loop regime, the quality range across reduction tools collapses $7\times$ (single-shot spread 0.42 $\rightarrow$ agent-loop spread 0.059); the agent rescues weak contexts via follow-up tool calls. However, cost differences persist: weak contexts force the agent to issue 2–$4\times$ more tool calls to recover. (3) A cross-family LLM-summary pair (gpt-5-mini summarizer feeding a Claude Haiku debugger) beats the same-family pair by $+0.071$ averaged across four diagnoser variants, falsifying the self-call-bias hypothesis on this task. The gpt-5-mini summarizer is also the agent-loop #1 method (score 0.749) at 0.37 tool-calls per case and $10\times$ lower reducer cost than the Haiku summarizer ($0.18 vs $1.75 per case). All data, code, per-case bundles, and reproducibility infrastructure are public.

1 Introduction

LogDx-CI evaluates **whether a log reduction tool preserves enough evidence for an LLM to identify the root cause of a CI failure**. The pipeline is:



The corpus is real public GitHub Actions failures across 8 categories and 7+ ecosystems. Ground truth is AI-drafted (Claude Opus 4.7) + single-author verified.

Why this matters. CI failure logs in this corpus range from 27 lines to 200k+, with a median around 5k. Most exceed the effective input window of even long-context models once tool definitions, system prompts, and reasoning overhead are accounted for. A reduction step is therefore nearly mandatory in production agent stacks (Claude Code, Codex, Cursor all ship some form of it), but the field has had no public empirical comparison of which reductions preserve enough evidence for downstream LLM diagnosis. This benchmark closes that gap.

Preprint.

Scope. This is a **niche benchmark**, not a general agent evaluation. It does not measure: general coding ability, multi-step planning, repository navigation, or open-ended debugging. Findings should not be extrapolated beyond "single-LLM-call or 5-turn-agent diagnosis of a single CI failure log."

2 Related Work

Coding-agent benchmarks. The dominant benchmark for LLM-based software engineering is SWE-bench Jimenez et al. [2024], which measures whether an agent can resolve real GitHub issues by editing a repository. Terminal-Bench Stanford and Laude [2024] measures agent task completion in a terminal environment, a closer analog to our CI-failure-diagnosis setting but optimizing for end-to-end task success rather than diagnostic accuracy. WebArena Zhou et al. [2024] and OSWorld Xie et al. [2024] benchmark agents in web and OS environments respectively. None of these benchmarks isolate the **context-reduction step** that precedes LLM diagnosis—the upstream tool that selects what evidence reaches the model is treated as opaque infrastructure. We measure this step directly.

LLM-as-judge evaluation. Many recent benchmarks rely on LLMs to score outputs: MT-Bench and Chatbot Arena Zheng et al. [2023], AlpacaEval Li et al. [2023], G-Eval Liu et al. [2023], and RAGAS Es et al. [2024] for retrieval-augmented generation. Several biases complicate this approach. Zheng et al. Zheng et al. [2023] document **self-enhancement bias** (models prefer their own outputs in pairwise comparison) and **position bias** (the first response wins in pairwise judging disproportionately often). Our Section 4.5 measurement extends this literature with a different bias variant: whether a summarizer plus downstream judge of the same vendor family produces inflated scores via shared priors. We find this **family-self-call bias is not present** on our task; cross-family pairs beat same-family by $+0.071$ on average across diagnoser variants.

Log compression and parsing. Classical log-parsing work (Drain He et al. [2017], Spell Du and Li [2016], LogPAI and LogHub Zhu et al. [2019]) optimizes log compression for *storage and human search*. Our work differs in the objective function: we measure how much of the failure signal survives various log-reduction strategies as judged by downstream LLM diagnostic ability, rather than measuring compression ratio or human-search recall. The RTK rtk-ai [2024] tool, which is a primary baseline in our benchmark, is an open-source context-reduction CLI without a published benchmark of its CI-debugging effectiveness—this benchmark is, in part, that missing measurement.

Context selection for LLMs. A growing literature studies how LLM performance degrades with context size and content structure: Lost-in-the-Middle Liu et al. [2024] shows that LLMs underweight information in the middle of long inputs. Self-RAG Asai et al. [2024] adds reflection to retrieval-augmented generation. Our hybrid routers—the top-2 baselines on this benchmark—are a particularly simple instance of this design space: a 120k-token threshold rule that empirically identifies the abstain cliff for Sonnet 4.6 and Haiku 4.5 on this corpus, with a deterministic fallback to `tail-200`. The $7\times$ quality-range collapse in agent-loop (Section 4.4) is consistent with recent findings that tool-use rescues weak retrieval in agent settings.

3 Methodology

3.1 Corpus

35 real GitHub Actions failure cases across 6 splits (Table 1): 8 failure categories (`test_assertion`, `compile_error`, `type_error`, `lint_failure`, `dependency_install`, `docker_build`, `timeout_or_oom`, `multi_failure`), 7+ ecosystems (pytest, cargo, `go test`, Maven, pnpm+jest, docker buildx, helm/k8s, etc.), and log sizes from 27 lines to 200k+.

Each case carries five files under `cases/<split>/<case_id>/`: `raw.log`, `case.json` (metadata), `ground_truth.json` (root cause, required signals, relevant files/tests, must-mention checklist, forbidden claims), `tags.json`, and `privacy_audit.json`.

Logs are sourced from publicly visible GitHub Actions runs. Each passed through a privacy audit (200k-line cap, fail-closed on truncation, URL / bearer / API-key / long-opaque-token redactors) before commit. Zero redaction hits across all 35 cases.

Table 1: Corpus splits.

Split	Cases	Wave
dev	5	v1 (prototype)
holdout	5	v1 (prototype)
stress	6	v1 (prototype)
v2/dev	3	v2 (formal)
v2/holdout	10	v2 (formal)
v2/stress	6	v2 (formal)
Total	**35**	

3.2 Evaluation metric

The primary metric `diagnosis_score_v1_1` is a calibrated linear combination of category accuracy, required-signal recall, relevant-file and relevant-test recall, must-mention coverage from the ground-truth checklist, and valid-evidence-quote rate. Penalized for forbidden claims and confident errors (confidence ≥ 0.70 and demonstrably wrong on multiple fronts—the metric closest to "this method led the LLM to confidently misdiagnose").

We also report `confident_error_rate_v1_1` as a separate column because confidently-wrong diagnoses are operationally distinct from "missed the diagnosis"; the safety implications differ.

3.3 Baselines — 11 context providers

Table 2: 11 baseline context providers.

Provider	Implementation
raw	Full log handed to the model
tail-200	Last 200 lines
grep	Regex-filtered failure-pattern lines + 3/8 context
rtk-read	RTK in read mode
rtk-log	RTK in log mode
rtk-err-cat	RTK in error-category mode
llm-summary-v1-haiku	Real Haiku 4.5 map-reduce summarizer†
llm-summary-v1-gpt-5-mini	Real gpt-5-mini map-reduce summarizer†
hybrid-grep-4k-rtk-err-cat	Earlier 4k-threshold hybrid (replaced)
hybrid-grep-120k-tail	grep $\leq$ 120k tokens else tail-200
hybrid-grep-120k-rtk-tail	grep $\leq$ 120k else rtk-err-cat (if not truncated) else tail-200

The 120k threshold is empirically the abstain cliff for Sonnet 4.6 and Haiku 4.5 on this corpus; above it, the model context window plus diagnostic prompt overhead causes abstention. See Section 4.6 for the density-driven inflation failure mode that motivates the choice.

† All map-reduce summarizers use `chunk_lines=500`, `chunk_overlap_lines=25`, `temperature=0`. Three of the 35 cases used `chunk_lines=100` instead for the Haiku summarizer because they contained 500-line windows exceeding Haiku's effective input window after Claude-Code session overhead; recorded in per-case `metadata.chunk_lines`.

3.4 Diagnosers

Four diagnosers receive the same prompt template and produce the same diagnosis JSON schema. Outputs are evaluated by the same deterministic evaluator.

- `real-debugger-v1`: Claude Haiku 4.5, single-shot.
- `real-debugger-v2`: Claude Sonnet 4.6, single-shot.
- `real-debugger-v3`: OpenAI gpt-5-mini (pinned to `gpt-5-mini-2025-08-07`), single-shot.

- `real-agent-v1`: Claude Sonnet 4.6 plus four deterministic tools (`grep`, `read_file`, `tail`, `view_log_lines`) operating on the raw log. 5-turn cap, 180k cumulative-input cap.

The agent variant operates on the *raw log*: tool calls bypass the upstream context provider and let the agent re-query the underlying log when the initial reduction is insufficient. This separates "context quality" from "agent recovery capability."

4 Results

4.1 Single-shot headline

Table 3 reports `diagnosis_score_v1_1` case-count-weighted macro across the 35-case corpus, aggregated over the three single-shot debugger families.

Table 3: Single-shot leaderboard. `conf_err` is the rate of confidently-wrong diagnoses (lower is better).

Rank	Method	Haiku	Sonnet	gpt-5-mini	Overall	conf_err	tokens/case
1	`hybrid-grep-120k-rtk-tail`	0.624	0.679	0.706	**0.670**	0.000	19,844
2	`hybrid-grep-120k-tail`	0.610	0.730	0.658	**0.666**	0.010	19,753
3	`llm-summary-v1-gpt-5-mini`	0.654	0.686	0.652	**0.664**	0.010	537,638
4	`grep`	0.578	0.684	0.655	0.639	0.000	88,355
5	`llm-summary-v1-haiku`	0.583	0.704	0.608	0.632	0.029	1,681,520
6	`tail-200`	0.595	0.624	0.623	0.614	0.019	6,108
7	`hybrid-grep-4k-rtk-err-cat`	0.552	0.597	0.571	0.573	0.029	19,892
8	`rtk-err-cat`	0.455	0.488	0.467	0.470	0.029	19,850
9	`raw`	0.324	0.368	0.367	0.353	0.000	275,248
10	`rtk-read`	0.329	0.369	0.349	0.349	0.010	274,289
11	`rtk-log`	0.238	0.262	0.249	0.249	0.133	810

Three findings from Table 3: (i) Both Pareto-winning hybrids beat every single-method baseline on quality. The $+0.03$ to $+0.04$ gap over `grep` is modest, but the token-cost axis is $4.5\times$ cheaper. (ii) The top-3 methods produce zero or near-zero confidently-wrong diagnoses. `rtk-log` misleads a confident LLM on $\sim$13 % of cases. (iii) `grep` is dominated by `hybrid-grep-120k-tail` on both axes: same-ballpark score (0.639 vs. 0.666) at $4.5\times$ fewer tokens (88,355 vs. 19,753).

4.2 Cross-debugger stability

Table 4 shows the top-3 method set under each single-shot debugger family.

Table 4: Top-3 sets per debugger family.

Family	Top-3
Claude Haiku 4.5	`hybrid-grep-120k-rtk-tail`, `hybrid-grep-120k-tail`, `tail-200`
Claude Sonnet 4.6	`hybrid-grep-120k-tail`, `grep`, `hybrid-grep-120k-rtk-tail`
OpenAI gpt-5-mini	`hybrid-grep-120k-rtk-tail`, `hybrid-grep-120k-tail`, `grep`

The **top-3 intersection** across all three families is {`hybrid-grep-120k-rtk-tail`, `hybrid-grep-120k-tail`} —both 120k-threshold hybrids. The **bottom-4 set** is also stable across all three families: {`raw`, `rtk-read`, `rtk-log`, `rtk-err-cat`}. The cross-family agreement is a direct robustness check; the direction is preserved across two vendors (Anthropic + OpenAI) and two within-Anthropic capability tiers (Haiku + Sonnet).

4.3 Cost-quality Pareto frontier

Figure 1 visualizes the cost-quality trade-off. The 4-method Pareto frontier is `rtk-log` (810 tokens, 0.249) $\rightarrow$ `tail-200` (6,108, 0.614) $\rightarrow$ `hybrid-grep-120k-tail` (19,753, 0.666) $\rightarrow$ `hybrid-grep-120k-rtk-tail` (19,844, 0.670). Every other method is dominated. Notable dominations: `grep` is dominated by `hybrid-grep-120k-tail` at $4.5\times$ fewer tokens; `rtk-err-cat` is dominated by `hybrid-grep-120k-tail` at similar token cost but $+0.20$ higher score; and `llm-summary-v1-haiku` is the most expensive method on the board at 1.68M tokens per case (predominantly Claude-Code-CLI nested cached-prefix overhead) but scores rank 5.

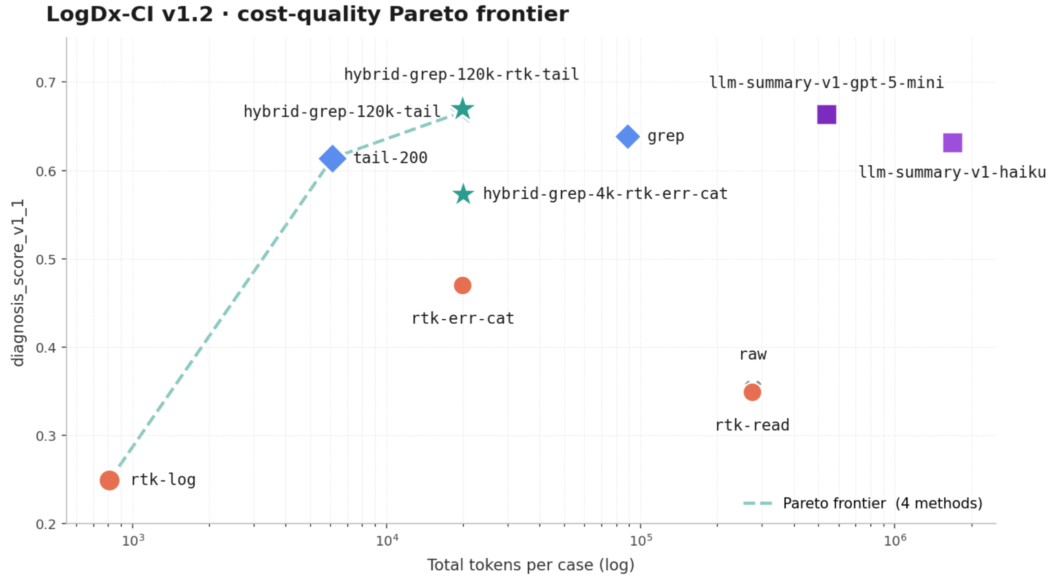


Figure 1: Cost-quality Pareto frontier. x-axis is total tokens per case (log scale); y-axis is `diagnosis_score_v1_1`. Green dashed line traces the 4-method frontier (`rtk-log` → `tail-200` → `hybrid-grep-120k-tail` → `hybrid-grep-120k-rtk-tail`). All other methods are dominated.

Pinned USD costs (per case, snapshot 2026-05-20): top-2 hybrids \$0.031, `tail-200` \$0.012, `grep` \$0.129, `llm-summary-v1-gpt-5-mini` \$0.184, `raw` \$0.392, `llm-summary-v1-haiku` \$1.760.

4.4 Agent-loop measurement

The single-shot leaderboard tests `log` → `reducer` → `single LLM call` → `answer`. Real Claude Code / Codex usage looks different: the model can call follow-up tools when its initial context is missing something. We add the agent-loop measurement using `real-agent-v1` (Sonnet 4.6, 5-turn cap, 4 deterministic tools).

Table 5: Agent-loop leaderboard.

Rank	Method	Single-shot	Agent	Δ	conf_err	Tools/case
1	`llm-summary-v1-gpt-5-mini`	0.664	**0.749**	+0.085	0.000	0.37
2	`hybrid-grep-120k-rtk-tail`	0.670	0.747	+0.077	0.000	0.97
3	`hybrid-grep-4k-rtk-err-cat`	0.573	0.737	+0.164	0.000	1.40
4	`hybrid-grep-120k-tail`	0.666	0.735	+0.069	0.000	1.00
5	`rtk-read`	0.349	0.735	+0.386	0.000	1.46
6	`grep`	0.639	0.722	+0.083	0.029	1.20
7	`tail-200`	0.614	0.710	+0.096	0.029	0.69
8	`rtk-err-cat`	0.470	0.708	+0.238	0.000	1.66
9	`llm-summary-v1-haiku`	0.632	0.690	+0.058	0.057	0.71
10	`rtk-log`	0.249	0.689	+0.440	0.057	2.60
11	`raw`	0.353	0.688	+0.335	0.029	1.68

Findings: (i) The quality range collapses 7×. Single-shot spread is 0.42 (0.670 – 0.249); agent-loop spread is 0.059 (0.749 – 0.690). The agent rescues weak contexts via tool calls; `rtk-log` gains +0.440, `rtk-read` gains +0.386, `raw` gains +0.335. (ii) Safety mostly collapses. Five of eleven methods sit at 0 % confident-error in agent-loop. (iii) The top single-shot method holds in agent-loop: `hybrid-grep-120k-rtk-tail` is agent-loop #2 at 0.747, just behind `llm-summary-v1-gpt-5-mini` at 0.749 (within Sonnet temperature-0 variance), using only 0.97

tool calls per case. (iv) `llm-summary-v1-gpt-5-mini` uses the lowest tool count of any method (0.37 / case — half as many as `tail-200`'s 0.69). The real gpt-5-mini summary front-loads the failure signal so completely that the agent commits to a diagnosis on turn 1 about 60 % of the time.

4.5 Cross-family LLM-summary

A reviewer raised after v1.1: was the haiku-summary's headline promotion anchored on Claude-family priors? Specifically, does the self-call pair (Haiku-summarizer → Haiku-debugger) carry shared prior bias that inflates its score?

v1.2 backfills a non-Anthropic summarizer (`llm-summary-v1-gpt-5-mini`—real OpenAI gpt-5-mini map-reduce, same prompts / `chunk_lines` / temperature as the Haiku summarizer) across the full 35-case × 4-diagnoser matrix (Table 6).

Table 6: Haiku summarizer vs. gpt-5-mini summarizer, paired with each diagnoser.

Diagnoser	Haiku-sum	gpt5mini-sum	Δ
`real-debugger-v1` (Haiku 4.5)	0.583	**0.654**	+0.071
`real-debugger-v2` (Sonnet 4.6)	**0.704**	0.686	−0.018
`real-debugger-v3` (gpt-5-mini)	0.608	**0.652**	+0.044
`real-agent-v1` (Sonnet+tools)	0.690	**0.749**	+0.059

Cross-family beats self-pair in 3 of 4 diagnosers. Specifically: Haiku-summary is best on the Sonnet debugger (not on Haiku), and gpt-5-mini-summary is best on the Haiku debugger (not on gpt-5-mini). There is no clean "summarizer-and-debugger of the same family" advantage. Summary quality is driven by the summarizer's ability to extract failure signal at chunk granularity, not by shared priors between summarizer and downstream debugger model families. The self-call-bias hypothesis is **falsified** for this task.

The cost gap matters too. The gpt-5-mini summarizer costs 10× less per reducer call than haiku-summary ($0.18 vs $1.75 per case). The gap is Claude-Code-CLI nested-invocation overhead (cached-prefix tokens) that the OpenAI-direct call doesn't carry.

4.6 Failure modes

Two reproducible failure modes worth naming:

Density-driven context inflation for `grep` / `rtk-err-cat`. Logs where `error|failed` markers appear in test-progress noise cause `grep` / `rtk-err-cat` outputs to exceed the model's effective reasoning budget. The rust compiletest (31k lines → 161k tokens after grep) and a nodejs timeout case (10k lines → 359k tokens) both push Sonnet and Haiku into abstain. `tail-200` survives by being content-blind and bounded; this motivates the 120k hybrid threshold.

RTK truncation drops bounded failure structure. `rtk-err-cat`'s aggressive compression strips assertion diffs, snapshot diffs, and structured compiler error blocks that the diagnoser needs. On 5 of 8 v2-formal cases where the legacy 4k-threshold hybrid routed to `rtk-err-cat`, `grep` would have done strictly better. This is the v1.3-prototype "selection-by-method" overfit that the v1.2 120k-threshold hybrids correct.

5 Recommendations

Three takeaways for practitioners deploying LLM-based CI debugging:

1. **Single-LLM-call diagnosis (no tool use):** use `hybrid-grep-120k-rtk-tail`. Top-1 across all three model families, zero confident-error, $0.03 per case, 4.5× cheaper than standalone `grep`.
2. **Tool-using agents (Claude Code, Codex-style):** `llm-summary-v1-gpt-5-mini` is the v1.2 default. Agent-loop #1 at 0.749, lowest tool-call count (0.37 / case), $0.18 / case end-

to-end. The hybrid above is a close second (0.747, 0.97 tools per case) and is appropriate when an extra LLM preprocessing call is not acceptable.

3. **Avoid `rtk-log` standalone.** Its 13.3 % confident-error rate (single-shot) means it actively misleads downstream LLMs $\sim$1 in 8 cases.

What this benchmark *does not* settle: configured / tuned RTK performance (v1.2 tests stock invocations only), generalization to non-Anthropic / non-OpenAI model families, performance on log shapes outside the 35-case distribution (notably pre-step CI runner output, build-system streams, monorepo matrix jobs without a single failing leaf).

6 Caveats and limitations

This is a **v1.2 preprint**. Headline findings are robust enough to ship; per-case magnitudes are preliminary.

1. **35 cases.** Per-case variance can shift macro means by ± 0.05 with future corpus expansion. The direction of the top-3 $\cap$ finding is robust across debugger families; absolute magnitudes are preliminary.
2. **Ground truth is AI-drafted (Claude Opus 4.7) + single-author verified.** Not independent human annotation. The full 35-case set has not been re-scored by an outside party.
3. **Three model families tested** (Anthropic Haiku 4.5 + Sonnet 4.6; OpenAI gpt-5-mini; OpenRouter Sonnet 4.6 for the agent-loop diagnoser). Two unique vendors. Adding Gemini, Llama, or DeepSeek is the most-leveraged follow-up.
4. **gpt-5-mini reproducibility caveat.** gpt-5-mini exhibits run-to-run variance even at `temperature=0` (reasoning models sample reasoning traces freshly each call). Practical effects: macro means in the leaderboard tables are stable to ± 0.02 across re-runs; per-case byte-identical reproduction is not guaranteed. The aggregate ranking is robust; specific cell numbers may shift by ± 0.02.
5. **Agent-loop soft-cap.** The 5-turn, 180k-cumulative input cap is soft. Despite guards, 18 of 350 v1.1 rows landed above 180k (max 273,654). Costs reported reflect actual usage, not the nominal cap.
6. **20 historical exclusions** documented in `configs/historical_provider_error_exclusions.json` appear as zero-score abstentions in the eval denominator. These correspond to transient model / CLI / API failures during the 2026-04 / 05 prototype sweeps, removed by the 2026-05-15 cleanups.
7. **Hybrid threshold is the variable, not the method shape.** This benchmark cannot say "hybrid as a strategy is bad." The v1.2 result says "the 120k-token threshold tuned on this 35-case corpus is the empirical abstain cliff for Sonnet 4.6 and Haiku 4.5; a fresh corpus might calibrate to a different threshold."

7 Reproducibility

Every release carries: (i) a **protocol lock** (SHA-pins 10 schemas + 3 prompts + 4 evaluators + 10 baselines + 35 case files at the release commit); (ii) **3 release gates** that fail CI when any committed artifact drifts; (iii) a **165-test suite** covering unit, integration, and end-to-end paths; (iv) **cache identity validation** (`metadata.diagnoser_config_sha256` and `metadata.shim_sha256` on every fresh row); and (v) **secret redaction** via URL / bearer / API-key / long-opaque-token / hostname redactors.

Reproducing a published number.

```
git clone https://github.com/eyuansu62/LogDx.git
cd LogDx && git checkout v1.2
python3 tools/validate_protocol_lock.py \
    --protocol protocols/logdx-ci-v2-partial-2026-05-20.lock.json
python3 tools/validate_committed_diagnosis_provider_errors.py
python3 tools/validate_eval_manifest_consistency.py
```

```
python3 tools/validate_diagnosis_vs_context_consistency.py
python3 tools/evaluate_diagnosis.py --split v2/dev \
    --diagnoser real-debugger-v3
```

The cases corpus is mirrored at huggingface.co/datasets/eyuansu71/logdx-ci with a flat-metadata viewer for HF Dataset Viewer browsing. For the full per-case bundle (`raw.log` + `ground_truth` + `tags` + `privacy_audit`), fetch via `huggingface_hub.snapshot_download`.

8 Conclusion

LogDx-CI v1.2 is the first public benchmark of CI log reduction tools optimized for downstream LLM diagnostic accuracy across model families. The three load-bearing findings (hybrid Pareto dominance, 7× agent-loop quality-range collapse, falsified self-call bias) carry concrete implications for production coding-agent stacks: at $0.03 per case for single-shot diagnosis and at $0.18 per case with a cross-family LLM-summary prefilter for agent-loop diagnosis, the cost of getting context-reduction right is at least 10× less than the cost of getting it wrong. Future work includes corpus expansion (target 50+ cases), additional model families (Gemini, Llama, DeepSeek), configured-RTK baselines, and independent third-party re-scoring of the 35-case ground-truth set.

Acknowledgments

LogDx-CI benchmarks third-party log-reduction tools alongside its own baselines. The `rtk-read`, `rtk-log`, and `rtk-err-cat` baselines are three different invocations of RTK rtk-ai [2024] by rtk-ai. CI failure logs are sourced from publicly visible GitHub Actions runs. Diagnoses are produced by Claude (Anthropic) and gpt-5-mini (OpenAI).

References

Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.11511.

Min Du and Feifei Li. Spell: Streaming parsing of system event logs. In *IEEE International Conference on Data Mining (ICDM)*, pages 859–864, 2016.

Shahul Es, Jithin James, Luis Espinosa-Anke, and Steven Schockaert. RAGAS: Automated evaluation of retrieval augmented generation. In *Conference of the European Chapter of the Association for Computational Linguistics (EACL): System Demonstrations*, 2024. URL https://arxiv.org/abs/2309.15217.

Pinjia He, Jieming Zhu, Zibin Zheng, and Michael R. Lyu. Drain: An online log parsing approach with fixed depth tree. In *IEEE International Conference on Web Services (ICWS)*, pages 33–40, 2017. doi: 10.1109/ICWS.2017.13.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2310.06770.

Xuechen Li, Tianyi Zhang, Yann Dubois, Rohan Taori, Ishaan Gulrajani, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. AlpacaEval: An automatic evaluator of instruction-following models. https://github.com/tatsu-lab/alpaca_eval, 2023.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics (TACL)*, 12:157–173, 2024. URL https://arxiv.org/abs/2307.03172.

Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2303.16634.

rtk-ai. RTK: Rust token killer. https://github.com/rtk-ai/rtk, 2024. Open-source CLI for context reduction. Primary baseline in this benchmark.

Stanford and Laude. Terminal-Bench: A benchmark for agent task completion in the terminal. https://www.tbench.ai/, 2024. Stanford × Laude collaboration. Task creators include Nicholas Carlini and Jan-Lucas Uslu. No formal published paper at fetch time; cite as software/benchmark.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2404.07972.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023. URL https://arxiv.org/abs/2306.05685.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.13854.

Jieming Zhu, Shilin He, Jinyang Liu, Pinjia He, Qi Xie, Zibin Zheng, and Michael R. Lyu. Tools and benchmarks for automated log parsing. In *IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, pages 121–130, 2019. LogPAI / LogHub.